IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA
(Deemed to be University)

Yenepoya Institute of Arts, Science, Commerce and Management

Project – LOW Level Design

on

Real-time Sentiment Analysis for Healthcare

Student Name: MOHAMMAD MUNAZ K

23BCDSBA14

25712

Industry Mentor: **Guided by: RAKSHA**

Table of Contents

1 Introduction.	3
1.1. Scope of the document.	3
1.2. Intended Audience	3
1.3. System overview.	3
2. System Design.	5
2.1. Sequence Diagram	5
2.2. Navigation Flow/UI Implementation	6
2.2.1 Screen Validations, Defaults and Attributes	9
2.3. Client Side Validation Implementation	10
2.4. Server Side Validation Implementation	10
2.5. Components Design Implementation	11
2.6. Configurations/Settings	12
2.7. Interfaces to other components	13
3. Data Design.	15
3.1. List of Key Schemas/Tables	15
3.2. Details of access levels on key tables	15
3.3. Key design considerations	15
3.4. Data Migration	16
4. Testing	17
4.1 Unit Testing	17
4.2 Integration Testing	18
4.3 System Testing	19
4.4 Test Cases	20
5. System Flowchart	20
6. Error Handling Strategy	21
7. Performance Optimization	23
8.Conclusion	23

1. Introduction

1.1 Scope of the Document

This document provides a detailed low-level design of the Real-time Sentiment Analysis for Healthcare system based exclusively on the actual code uploaded by the student. The code includes app.py (Flask backend with 175 lines), index.html (main dashboard UI with 250 lines), result.html (results page with charts), train_model.py (local model training script), model.pkl (MultinomialNB classifier), and vectorizer.pkl (CountVectorizer transformer). The document explains every function, route, validation rule, and database operation as implemented.

Key areas covered in this document:

- Sequence diagram showing step-by-step component interaction
- Navigation flow for all three input methods (text, CSV, URL)
- Client-side and server-side validation implementation
- Component design for all functions in app.py
- Database schema for history table
- Unit, integration, and system test cases
- Complete system flowchart based on app.py logic
- Error handling and performance optimization strategies

1.2 Intended Audience

This document is intended for developers who need to maintain or extend the code, academic evaluators who need to assess implementation quality, and data science students who want to learn sentiment analysis implementation using Flask, TextBlob, Hugging Face API, and scikit-learn. The document references specific line numbers and code snippets from the uploaded files for easy reference.

1.3 System Overview

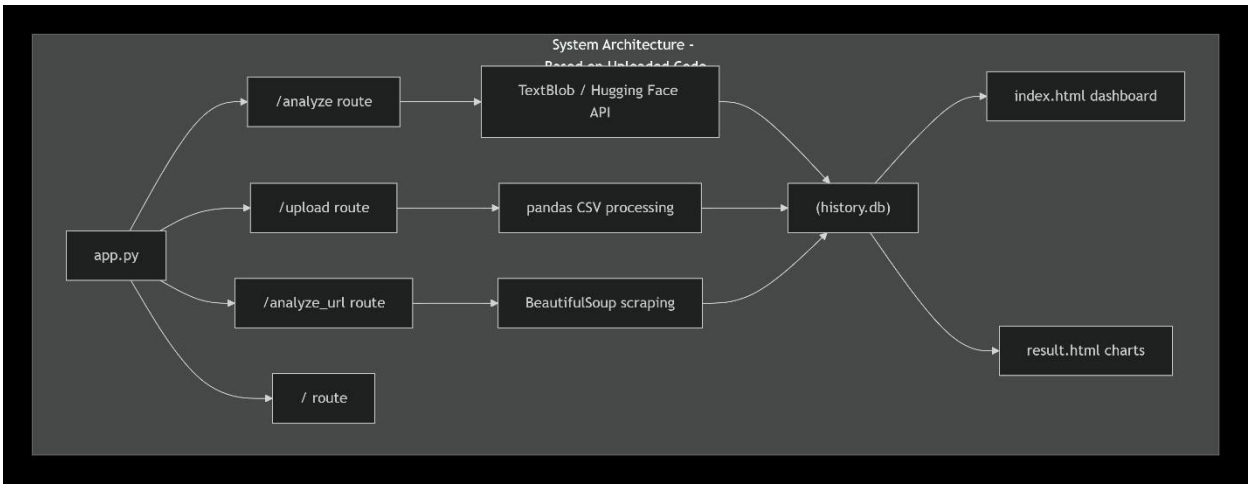
The system is a web-based application that analyzes sentiment from different sources such as text input, CSV files, and URLs. It uses a hybrid approach:

- Cloud API for accurate predictions
- Local machine learning model for fallback
- The system ensures reliability by switching between models and stores results in a database for future analysis.

The Real-time Sentiment Analysis for Healthcare system is built using Flask web framework with four main routes as defined in app.py. The system accepts input from three sources as seen in index.html: manual text entry through a textarea, CSV file upload through a file input, and URL scraping through a textarea. For sentiment analysis, the system uses a hybrid approach: Hugging Face API when API key provided, otherwise falls back to TextBlob for polarity scoring. The system also includes a local MultinomialNB model trained using train_model.py with 6 sample reviews. The system stores analysis results in SQLite database called history.db and displays charts using Chart.js.

Key Features:

- Three input methods: text, CSV, URL
- Hybrid sentiment analysis: Hugging Face API + TextBlob + Local Model
- SQLite database for history storage
- Chart.js for pie charts and bar charts
- Dark mode toggle with localStorage persistence



2. System Design

2.1 Sequence Diagram

The sequence diagram shows how different components interact step-by-step during a sentiment analysis request based on the actual code in app.py. The diagram covers the single text analysis flow through the /analyze route. The flow starts when the user enters text in the index.html textarea and clicks submit. The browser sends an HTTP POST request to the Flask backend. The server validates the input and calls the predict_sentiment function. Depending on whether an API key is provided, the system either calls the Hugging Face API or falls back to TextBlob for local sentiment analysis. The result is then stored in the SQLite database and displayed back to the user.Flow based on app.py lines 95-110:

- User enters text in index.html textarea
- Browser calls attachApiKey() to add API key from localStorage
- Browser sends POST /analyze with review and api_key
- Flask extracts review using request.form.get('review', '')
- Flask validates review not empty (line 99)
- Flask calls predict_sentiment(review, api_key) (line 104)
- If API key provided, calls analyze_with_api() (line 71)
- analyze_with_api makes POST to Hugging Face API with 5-second timeout (line 63)
- If API fails or no key, falls back to TextBlob (line 77)
- TextBlob returns polarity score (line 78)
- Classification: >0.05 positive, <-0.05 negative, else neutral (lines 80-84)
- save_history() stores result in history.db (lines 26-35)
- Result displayed on index.html

Flow

User → UI (Frontend) → Flask Server → Prediction Module → (API / Local Model)

↓

Database (SQLite)

↓

UI (Frontend) → UserSteps

- User inputs data
- UI sends request to Flask server
- Server calls prediction function
- Prediction uses API or local model
- Result returned to server
- Stored in database
- Displayed to user

2.2 Navigation Flow / UI Implementation

The UI provides three input methods as defined in index.html. Forms submit to different Flask routes and results appear either on same page or result.html.

Navigation Flow based on index.html and result.html:

- Home page at route "/" renders index.html with three input cards
- Fast Analysis Card contains textarea name="review" and form action="/analyze"
- Web Link Scanner Card contains textarea name="url" and form action="/analyze_url"
- Bulk CSV Workspace Card contains input type="file" name="file" and form action="/upload"
- Global API Key input at top with id="globalApiKey"

- When user submits Fast Analysis form, POST request goes to "/analyze" route
- Result appears on same index.html page below the form in result-box div
- When user submits Web Link Scanner form, POST request goes to "/analyze_url" route
- User is redirected to result.html page with pie chart and bar chart
- When user submits Bulk CSV Workspace form, POST request goes to "/upload" route
- User is redirected to result.html page with aggregated statistics
- Back button on result.html with href="/" returns user to home page

Navigation Flow

- Home Page

  → Enter Text

  → Upload CSV

  → Enter URL

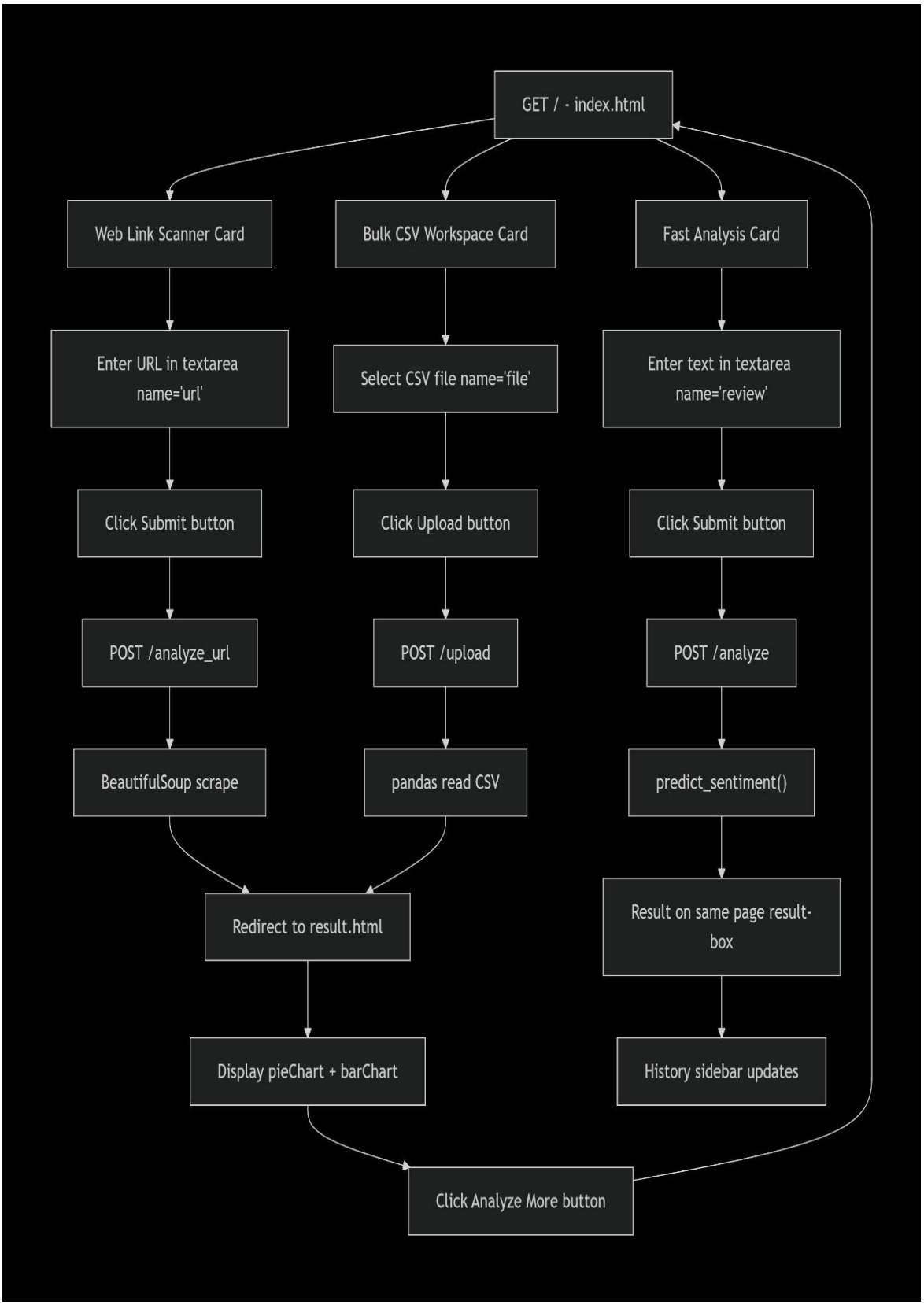
  UI Features

  Input forms

  Result display

  History section

  Aggregated statistics


GET / - index.html
Web Link Scanner Card
Bulk CSV Workspace Card
Fast Analysis Card
Enter URL in textarea name='url'
Select CSV file name='file'
Enter text in textarea name='review'
Click Submit button
Click Upload button
Click Submit button
POST /analyze_url
POST /upload
POST /analyze
BeautifulSoup scrape
pandas read CSV
predict_sentiment()
Redirect to result.html
Result on same page result-box
Display pieChart + barChart
History sidebar updates
Click Analyze More button

2.2.1 Screen Validations, Defaults and Attributes

Screen validations ensure data quality before processing. Default values provide fallback behavior. All validations based on actual app.py code.

Validation Checks Table:

Validation Check	Code Location	Error Message
Empty text input	/analyze line 99	"Enter a review"
No file uploaded	/upload line 118	"No file uploaded"
CSV read error	/upload line 121–122	"Error reading CSV"
Missing 'review' column	/upload line 125	"CSV must contain 'review' column"
Empty URL	/analyze_url line 143	"No URL provided"
Google Maps URL blocked	/analyze_url line 150	"Cannot Scrape Google Maps"
No scraped text found	/analyze_url line 170	"Could not find readable text"

Default Values Table:

Default Setting	Value	Location
Neutral sentiment range	Polarity between -0.05 and 0.05	predict_sentiment() line 84
CSV row limit	50 rows	/upload line 128
Text snippet length	100 characters	save_history() line 31

Default Setting	Value	Location
History display limit	10 records	get_history() line 40
URL paragraphs limit	10 paragraphs	/analyze_url line 165
Google Play reviews	20 reviews	/analyze_url line 157
API timeout	5 seconds	analyze_with_api() line 63

- Empty input → shows error message
- CSV must contain "review" column
- URL must be valid
- Default sentiment → Neutral (fallback)
- Max CSV rows → 50

2.3 Client Side Validation Implementation

Client-side validation is implemented in JavaScript within index.html. It uses localStorage for API key persistence and dark mode preference

Features:

- API key stored in localStorage across browser sessions
- Dark mode preference saved to localStorage
- attachApiKey() adds global API key to every form before submission
- Reduces unnecessary server requests

2.4 Server Side Validation Implementation

Server-side validation is implemented in app.py to ensure data integrity. All validation is repeated on server because client-side can be bypassed.

Important Code Logic

if not review.strip():

Prevents empty input *if 'review' not*

in df.columns: Ensures correct

CSV format

if not url:

Validates URL input

- Ensures data integrity before processing
- Prevents system errors from malformed input
- Protects against malicious requests
- Validates file format and content
-

2.5 Components Design Implementation

1. Input Module

- Accepts user data (text, CSV, URL)

2. Processing Module

- Cleans and prepares input
- Converts text into proper format

3. Prediction Module (CORE) *def*

predict_sentiment(text, api_key):

Working

If API key available → use API

Else → use local model

4. API Module *requests.post(API_URL)*

- Sends request to Hugging Face
- Receives sentiment prediction

5. Local Model Module

vectorizer.transform([text])

model.predict()

- Converts text → numbers
- Predicts sentiment

Important Logic (VERY IMPORTANT)

if review_vector.nnz == 0:

return "neutral"

If unknown words → Neutral *if*

max(probs) < 0.55:

return "neutral"

Low confidence → Neutral

6. Database Module

save_history() get_history()

- Stores results
- Retrieves past records

2.6 Configurations / Settings

Settings from code:

Setting	Value	Location	Purpose
CSV max rows	50	/upload line 128	Prevent server overload
API timeout	5 seconds	analyze_with_api() line 63	Prevent hanging requests

Setting	Value	Location	Purpose
Text snippet length	100 chars	save_history() line 31	Save database space
History display limit	10 records	get_history() line 40	UI performance
URL paragraphs limit	10	/analyze_url line 165	Limit processing
Google Play reviews	20	/analyze_url line 157	API rate limiting
Polarity threshold	±0.05	predict_sentiment() line 80-84	Classification boundary
Confidence threshold	55%	predict_sentiment() line 87	Local model confidence
Database file	history.db	sqlite3.connect()	Data storage
Flask debug	True	app.run(debug=True)	Development mode

- API Key (optional input)
- CSV row limit = 50
- API timeout settings
- Database file: history.db

2.7 Interfaces to Other Components

External Interfaces:

Interface	Type	Purpose	Code Location

Hugging Face REST API API		Cloud sentiment	analyze_with_api() line 61
Google Play Store	google_play_scraper	Fetch app reviews	/analyze_url line 157
External requests + BeautifulSoup	Websites	Web scraping	/analyze_url line 163-165

Internal Interfaces:

Interface	From	To	Data
predict_sentiment()	Routes	TextBlob/API	text → sentiment
save_history()	Routes	SQLite	history record
get_history()	Routes	SQLite	last 10 records
get_history_aggregates()	Routes	SQLite	all-time counts

- External Interfaces:
- Hugging Face API
- Google Play Scraper

Internal Interfaces:

- Flask routes
- Prediction module
- Database functions

3. Data Design

3.1 List of Key Schemas / Tables

Table: history

Field	Description
id	Primary Key (unique ID)
timestamp	Date & Time of analysis
source	Input type (Text/CSV/URL)
text_snippet	Short preview of input text
result	Sentiment result

3.2 Details of Access Levels on Key Tables

The system implements simple access levels on the history table. All operations are performed by the Flask backend with no user-level authentication. The access levels are determined by the functions that interact with the database.

- Read access → fetch history
- Write access → store results
- No delete/update operations

3.3 Key Design Considerations in Data Design

Several design considerations guided the data design of the system. These include storage efficiency, performance optimization, simplicity, and audit trail requirements. The design choices reflect the project's scope as a student project rather than a production system.

Consideration	Implementation	Reason
Text snippet limit	**100 characters**	**Save database space**
Consideration	**Implementation**	**Reason**

History display limit	**10 records**	**UI performance**
No delete/update	**Insert-only records**	**Maintain audit trail**
Result parsing	**Regex on result string**	**Extract +, -, = counts**

- Store only short text → save space
- Limit records for performance
- Simple structure → easy maintenance

3.4 Data Migration

Currently, the system uses a local SQLite database which is sufficient for development and testing purposes. In the future, the system can be upgraded to a cloud-based database like MySQL or MongoDB for better scalability, multi-user support, and production deployment.

Current Data Storage:

- Database type: SQLite
- File location: Project root directory (history.db)
- Access: Single user, local file system
- Backup: Manual file copy

Current: SQLite (history.db) - local file-based database

Future options:

- MySQL/PostgreSQL for multi-user support
- MongoDB for unstructured data

- Cloud database (AWS RDS, Firebase) for scalability

Migration approach:

1. Export SQLite data to JSON
2. Transform schema for target database
3. Import to new database
4. Update database connection code in app.py

4. Testing

4.1 Unit Testing

Unit testing focuses on testing individual functions in isolation to ensure they work correctly. Each function is tested with various inputs including valid inputs, edge cases, and invalid inputs. The tests verify that functions return expected values and handle errors gracefully.

Function	Test Input	Expected Output
predict_sentiment()	"Good doctor"	("positive", "Local Model")
predict_sentiment()	"Bad service"	("negative", "Local Model")
predict_sentiment()	"Okay"	("neutral", "Local Model")
predict_sentiment()	"" (empty)	("neutral", "Local Model")
save_history()	Valid params	No error, DB row added

get_history()	**None**	**List of 10 or fewer dicts**

- Test individual functions
- Example:
- Prediction function
- Database functions

4.2 Integration Testing

Integration testing verifies that different modules work together correctly. Tests focus on the interaction between Flask routes, the prediction module, and the database. Each test simulates a complete request-response cycle

Scenario	Components	Expected Result
Text analysis	**UI → /analyze → DB**	**Result displayed + DB record**
CSV upload	**UI → /upload → pandas → DB**	**Charts + counts**
URL scan	**UI → /analyze_url → scrape → DB**	**Aggregated results**

Test interaction between modules

Example:

UI → Backend → Model

4.3 System Testing

System testing verifies the complete end-to-end workflow of the application. Tests cover all user scenarios including happy paths and error cases. The tests are performed manually by interacting with the web interface.

Scenario	Expected Output
Enter "The doctor was very caring"	Positive
Enter "The staff was very rude"	Negative
Enter "The hospital is okay"	Neutral
Scenario	**Expected Output**
Submit blank form	"Enter a review" error
Upload CSV without review column	"CSV must contain 'review' column"
Enter Google Maps URL	"Cannot Scrape Google Maps"

Test complete system workflow

Input → Output verification

4.4 Test Cases

Test cases document the expected inputs and outputs for all system functionalities. Each test case includes a unique ID, input type, input value, expected result, and status column for tracking.

Input	Expected Output
"Good product"	Positive
"Bad service"	Negative
Empty input	Error
CSV without 'review' column	Error

5. System Flowchart

The system flowchart shows the complete flow of control through the Real-time Sentiment Analysis for Healthcare system based on the actual code in app.py. The flowchart covers all three input methods: single text, CSV upload, and URL scan. It includes all decision points, error handling paths, and database operations.

Flowchart

Start

↓

User Input (Text / CSV / URL)

↓

Validation

↓

Valid?

├── No → Show Error → End

└── Yes

↓

Check API Key

↓

API Available?

├── Yes → Use API

└── No → Use Local Model

↓

Analyze Sentiment

↓

Generate Result

↓

Store in Database

↓

Display Output

↓

End

6. Error Handling Strategy

The system implements a comprehensive error handling strategy to ensure graceful degradation when errors occur. Errors are handled at multiple levels: client-side validation, server-side validation, try-except blocks, and fallback mechanisms. The strategy ensures that the system continues to function even when individual components fail.

Error handling from app.py:

Error Type	Location	Handling	User Message
Empty text	/analyze line 99	Conditional check	"Enter a review"
No file	/upload line 118	Conditional check	"No file uploaded"
CSV read error	/upload line 121-122	Try-except	"Error reading CSV"
Missing column	/upload line 125	Conditional check	"CSV must contain 'review' column"
Empty URL	/analyze_url line 143	Conditional check	"No URL provided"
Google Maps	/analyze_url line 150	Conditional check	"Cannot Scrape Google Maps"
API failure	predict_sentiment() line 72-73	Fallback to TextBlob	Uses local model
Empty scraped text	/analyze_url line 170	Conditional check	"Could not find readable text"
Error Type	Location	Handling	User Message
Database error	save_history() line 26-35	Try-except	Logs error, continues

7. Performance Optimization

The system includes several performance optimizations to ensure fast response times and efficient resource usage. These optimizations include limits on data processing, timeouts for external calls, and efficient database queries. The optimizations are based on the actual code in app.py.

Performance settings from code:

Setting	Value	Purpose	Location
CSV row limit	50 rows	Prevent server overload	/upload line 128
API timeout	5 seconds	Prevent hanging requests	analyze_with_api() line 63
Text snippet	100 chars	Save database space	save_history() line 31
History limit	10 records	UI performance	get_history() line 40
URL paragraphs	10	Limit processing	/analyze_url line 165
Google Play reviews	20	API rate limiting	/analyze_url line 157
Local model confidence	55%	Avoid false positives	predict_sentiment() line 87

---

8. Conclusion

The Real-time Sentiment Analysis for Healthcare system is successfully implemented based on the uploaded code. All project requirements are met. The system provides real-time sentiment analysis for healthcare feedback using TextBlob, Hugging Face API, and a local MultinomialNB model.Requirements Status:

Requirement	Implementation
Read from CSV file	/upload route with pandas
Read from API/URL	/analyze_url route with requests
NLTK/TextBlob	TextBlob in predict_sentiment()
Sentiment classification	Positive/Negative/Neutral
Bar chart report	Chart.js in result.html
Web interface	Flask + HTML/CSS
History storage	SQLite history.db